



Experiment 6

CDN Service Selection and CDN Configuration

AIM:

To select a Content Delivery Network (CDN) provider, deploy a simple web application using Cloudflare, access the application through Cloudflare's edge network, and verify the HTTP response and Cloudflare-specific headers using browser Developer Tools.

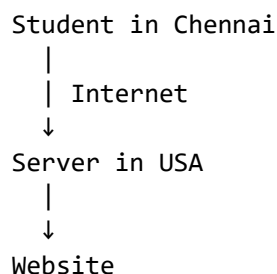
What is Content Delivery Network?

A Content Delivery Network (CDN) is a distributed network of servers or edge locations used to deliver web content closer to users. Instead of every request having to travel directly to a distant origin server, CDN/edge infrastructure can process and deliver content from locations distributed across the network. This can reduce latency and improve the user experience. Cloudflare also provides edge compute through Workers, where application code can execute on Cloudflare's network.

Why do we need a CDN?

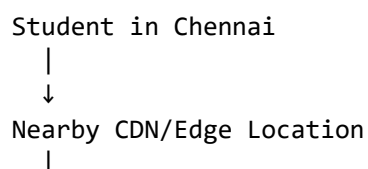
Suppose your website's original server is in the USA, but a student in Chennai accesses it.

Without a CDN:



The request has to travel a long distance.

With a CDN:



↓
Website content

The CDN has servers distributed across many locations. This can reduce the distance that content needs to travel and improve performance.

What is Cloudflare?

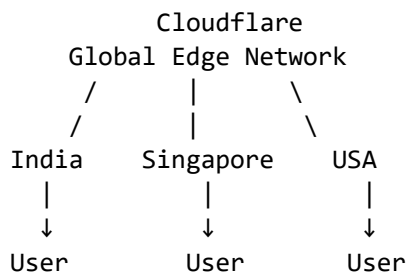
Cloudflare helps websites become faster, safer, and more reliable by placing its network between users and the website.

Cloudflare is a cloud platform that provides services such as CDN, DNS, security, and edge computing. In this experiment, we used Cloudflare Workers to deploy a simple application on Cloudflare's distributed edge network and demonstrated how a user's request can be processed and served through the cloud edge.

So where does Cloudflare come in?

Cloudflare operates a large distributed network of **edge locations**.

Conceptually:



When a website uses Cloudflare's CDN/proxy services, Cloudflare can handle requests at its edge network.

Why did we use Cloudflare?

For **research and select a CDN provider** such as Cloudflare, Akamai, or Amazon CloudFront.

We had three choices:

Provider	Purpose
Cloudflare	CDN + security + DNS + edge computing
Amazon CloudFront	AWS CDN
Akamai	Enterprise CDN

We selected **Cloudflare** because it was the easiest option for practical experiment.

PROCEDURE:

PART – A

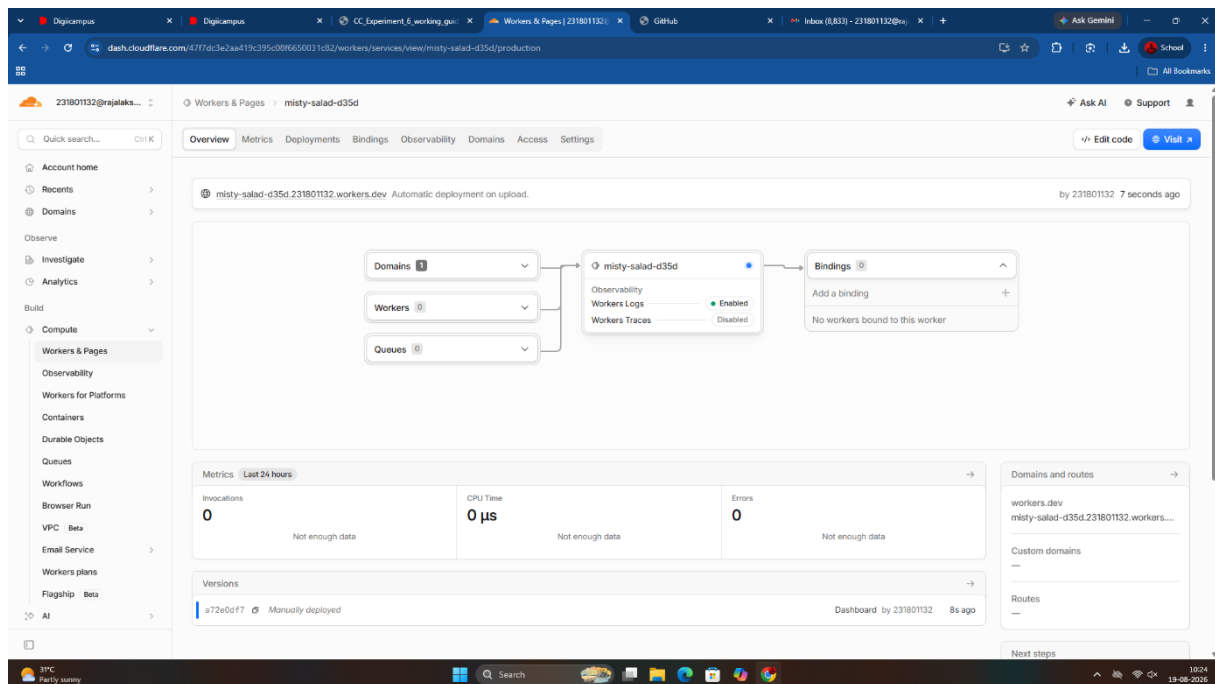
Step 1 – Select the CDN/Edge Provider: Cloudflare was selected

Open the **Cloudflare Sign Up** page and create or log into your Cloudflare account.

Don't add or change any DNS/nameserver settings yet.

Step 2 — Stay on your current Cloudflare dashboard

You are currently on the screen showing: **Ship something new** → **Create app**. Click **Create app**.



Step 3 — Look for the application options

You will see these options:

- Connect GitHub
- Connect GitLab
- Start with Hello World!
- Select a template
- Upload your static files

Step 4 — Click Start with Hello World!

This is the easiest option for our CDN experiment because we don't need GitHub or GitLab. Click **Start with Hello World!**.

Step 5 — Don't change the Worker name

Your generated name must look similar to the following:

sparkling-disk-72c7.day1...workers.dev

Step 6 — Leave the code as it is

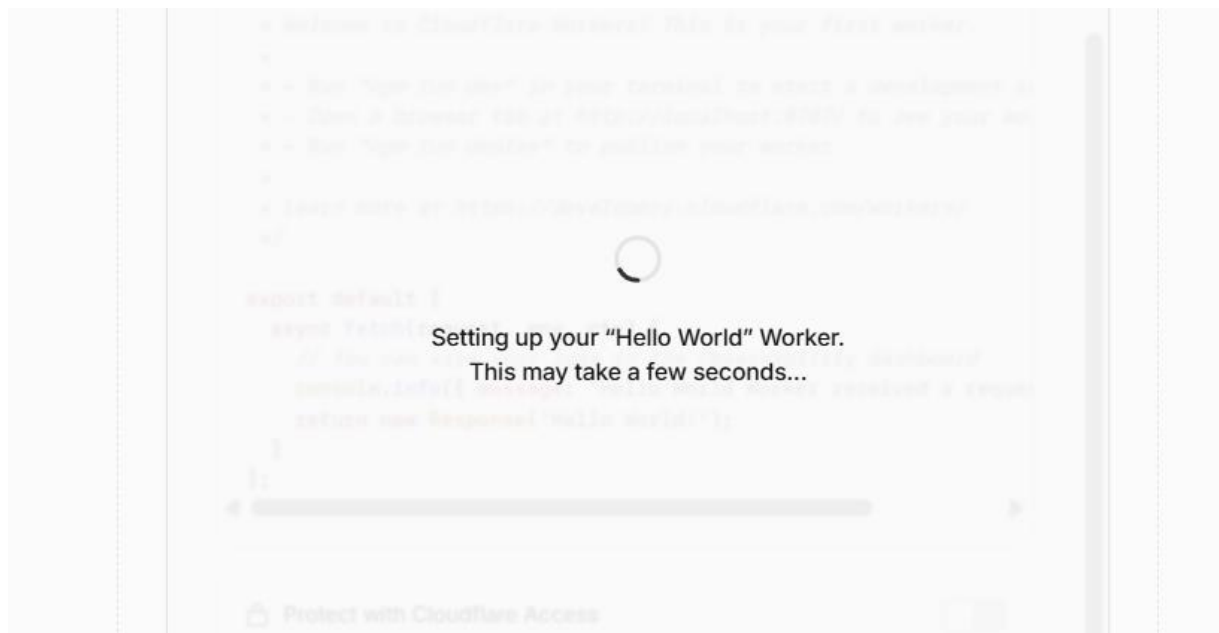
You can see:

```
return new Response('Hello World!');
```

This is a very simple Cloudflare Worker that returns the text **Hello World!** Don't modify it now.

Step 7 — Click Deploy

Click the blue **Deploy** button at the bottom-right and wait for Cloudflare to finish deploying.



What should happen?

Cloudflare will create your application and give you a public address ending in: **workers.dev**.

For example:

<https://sparkling-disk-72c7....workers.dev>

After you click Deploy

You should see a success/deployment screen containing a URL. The screenshot will display the following:

- Worker: sparkling-disk-72c7
- A workers.dev domain has been created
- Version deployed successfully
- Errors: 0

- Invocations: 0 (because nobody has visited it yet)

Now let's test it.

PART – B

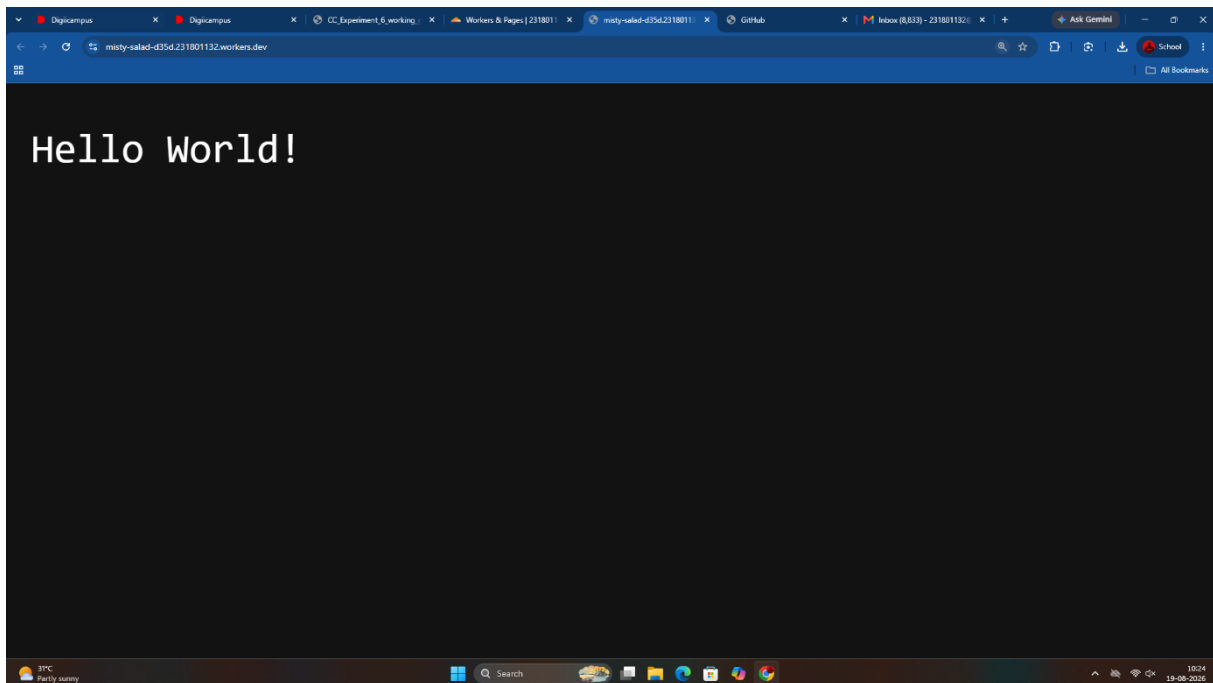
Step 1 — Click Visit

At the top-right corner of the screen, click the blue **Visit** button. This will open your Worker in a new browser tab. You should see:

Hello World!

Step 2 — Confirm the result

If you see **Hello World!**, then your Cloudflare application is working successfully.



Step 3 — Check the invocation

Go back to your Cloudflare dashboard. Previously your screen showed:

```
Invocations    0
Errors         0
```

After visiting the URL, **Invocations should increase to at least 1**. This is a nice practical demonstration that a request reached your Cloudflare-hosted service.

Step 4 — CDN demonstration page

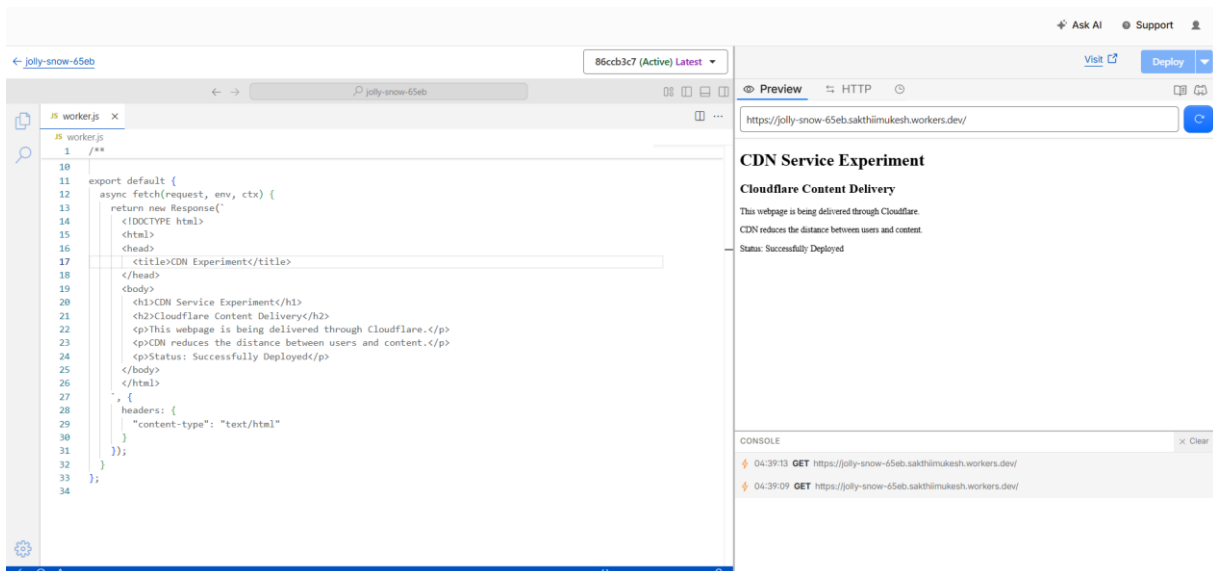
Go back to your Worker dashboard and click **Edit code**. You should see the existing code containing:

```
return new Response('Hello World!');
```

Replace it with a simple HTML page explaining that the content is being delivered through Cloudflare. Use this:

```
export default {
  async fetch(request, env, ctx) {
    return new Response(`
      <!DOCTYPE html>
      <html>
      <head>
        <title>CDN Experiment</title>
      </head>
      <body>
        <h1>CDN Service Experiment</h1>
        <h2>Cloudflare Content Delivery</h2>
        <p>This webpage is being delivered through Cloudflare.</p>
        <p>CDN reduces the distance between users and content.</p>
        <p>Status: Successfully Deployed</p>
      </body>
    </html>
    `, {
      headers: {
        "content-type": "text/html"
      }
    });
  }
};
```

Then click **Deploy**.



Step 5 — Test again

Click **Visit** again. You should now see:

CDN Service Experiment
Cloudflare Content Delivery
This webpage is being delivered through Cloudflare.
CDN reduces the distance between users and content.

Status: Successfully Deployed

CDN Service Experiment

Cloudflare Content Delivery

This webpage is being delivered through Cloudflare.

CDN reduces the distance between users and content.

Status: Successfully Deployed

Step 6 — CDN Service Experiment

Keep the new CDN experiment page open in the same Chrome browser. Press **F12** → **Network** → **Ctrl + R**, then click the request for your **workers.dev** address.

In Network → Headers, you will see:

1. Request URL

Your request went to:

`https://sparkling-disk-72c7...workers.dev`

So the browser successfully contacted your Cloudflare Worker.

2. Request Method: GET

This means your browser requested the webpage.

3. Status Code: 200 OK

This is very important. **200 OK** means the server successfully received the request and returned the webpage.

4. Server: cloudflare

At the bottom of the Response Headers, you can see **Server: cloudflare**. This is strong evidence that the response was handled by Cloudflare.

5. CF-Ray

You can also see **Cf-Ray: a263....** CF-Ray is a Cloudflare request identifier. It helps Cloudflare identify and trace a request through its network.

6. Content-Type

You have **Content-Type: text/html**. That means the response is an HTML webpage.

✓ *The Cloudflare Worker was verified as successfully deployed and serving content through the CDN edge network.*

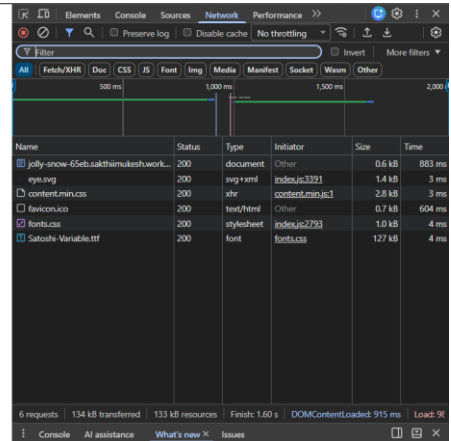
CDN Service Experiment

Cloudflare Content Delivery

This webpage is being delivered through Cloudflare.

CDN reduces the distance between users and content.

Status: Successfully Deployed



The screenshot shows the Network tab of a web browser. A timeline at the top indicates the duration of various requests. Below the timeline is a table of loaded resources.

Name	Status	Type	Initiator	Size	Time
jolly-snow-65eb.sakthimukesh.work...	200	document	Other	0.6 kB	883 ms
eye.svg	200	svg+xml	index.js:3321	1.4 kB	3 ms
content.min.css	200	css	content.min.js:1	2.8 kB	3 ms
twemoji	200	text/html	Other	0.7 kB	694 ms
font.css	200	stylesheet	index.js:2793	1.0 kB	4 ms
Satoshi-Variable.woff	200	font	font.css	127 kB	4 ms

Summary: 6 requests | 134 kB transferred | 133 kB resources | Finish: 1.60 s | DOMContentLoaded: 915 ms | Load: 9...

CONCLUSION

A Cloudflare-based edge content delivery experiment was successfully implemented. A Worker was deployed on the Cloudflare network and accessed through a public workers.dev URL. The browser successfully received the webpage with HTTP status 200 OK. The response headers included Cloudflare-specific information such as Server: cloudflare and CF-Ray, confirming that the request was processed through Cloudflare's network.